Latent Prediction Is Not Visual Robustness: Diagnosing the Invariance–Resolution Trade-off in JEPA World Models for Control

May 19, 2026

Abstract

Joint-Embedding Predictive Architectures (JEPAs) are commonly *believed* to learn abstract, invariant world representations: by predicting in latent space rather than reconstructing pixels, the encoder is expected to discard visual redundancy and noise. This expectation is a *community heuristic* rather than a published guarantee — to our knowledge no JEPA work formally claims pixel-noise robustness for control. We test the implicit assumption on **LeWorldModel (LeWM)** [13], a published JEPA world model, across four manipulation and navigation tasks (PushT, TwoRoom, Reacher, Cube) and eight levels of train-time pixel-noise augmentation. We find three things:

1. **Visual OOD collapse in JEPA + CEM control.** Without noise-aware training, LeWM collapses under mild pixel noise: PushT control success drops from 86.33% (clean) to 4.67% (Gaussian std = 0.08, near-random); TwoRoom drops from 94.00% to 50.00%.
2. **No global-optimal noise level exists.** Tasks respond very differently to noise augmentation: visually redundant TwoRoom prefers heavy noise (point-best at `std_max` = 0.008), while contact-heavy PushT peaks on *clean* at `std_max` = 0.003 but on *robustness* at `std_max` = 0.006 — clean and robust optima dissociate within a single task.
3. **A five-layer diagnostic protocol explains the compression mechanism.** Instrumenting encoder shift, encoder geometry, predictor sensitivity, latent-noise response, and task resolution traces noise-induced control failure to a representational chain: representation compression (drop in effective rank) → loss of transition-key resolution (`transition_resolution_ratio`) → loss of controllability (`id_probe_r`^{2}). However, used as a *cross-checkpoint* predictor, the strongest single diagnostic (`predictor_target_to_nn_cos_ratio_at_max_std`) tracks *overall ckpt quality on clean inputs*, not OOD-specific robustness. The diagnostic toolkit usefully ranks checkpoints by clean control fidelity but does not substitute for actually training with noise when OOD robustness is the goal.

This paper does not propose a new training algorithm. Its contribution is (i) a systematic empirical study of JEPA + CEM visual OOD failure, (ii) a reproducible diagnostic toolkit, and (iii) an honest delineation of what cross-ckpt diagnostics can and cannot predict.

Keywords: world models; JEPA; visual robustness; representation diagnostics; invariance–resolution trade-off.

1 Introduction

1.1 The implicit invariance heuristic and where it breaks

Since Yann LeCun proposed JEPA [12], the paradigm has been advanced as a direction for self-supervised learning. Unlike generative models (VAEs, diffusion), JEPA does not reconstruct pixels;

it predicts future *representations* in latent space. The motivating intuition — that predicting "what is invariant" rather than "what pixels look like" yields abstract representations that discard visual redundancy and noise — is now part of the field's informal vocabulary in talks, blog posts, and survey articles [1, 3].

We emphasise that this is a heuristic rather than a published guarantee. To our knowledge no JEPA paper has *formally claimed* visual-OOD robustness for control-relevant downstream tasks. I-JEPA [1] and V-JEPA [3, 2] established strong visual representations on ImageNet and video via masked prediction; LeWorldModel (LeWM) [13] extended the framework to end-to-end stable world-model training across four robotic control tasks. Existing robustness studies probe JEPA only on image classification [15], synthetic 1D distractors [10], or medical ultrasound [16]; none on *JEPA-based control* under realistic pixel noise.

This leaves a basic operational question open: **if the input image is degraded by sensor noise, lighting change, or camera jitter, does a JEPA + CEM world model still plan and act reliably?**

The data say no. On PushT (2D pushing), the untrained-with-noise LeWM achieves 86.33% on clean images but falls to 4.67% under Gaussian pixel noise of std = 0.08 — essentially random. TwoRoom (2D navigation) drops from 94.00% to 50.00%. Latent prediction alone does not, in this regime, confer the visual robustness the community heuristic would predict.

1.2 The core tension: no globally optimal noise level

A natural remedy is input-side noise augmentation during training, a technique long-validated in supervised and contrastive learning [5, 6]. But a deeper question arises: **does there exist a single, universal noise level that is optimal across tasks?**

A systematic sweep across four tasks at eight levels of `std_max` $\in \{0.001, \ldots, 0.008\}$ answers no:

- **TwoRoom** (visually redundant navigation): clean success rises monotonically with noise, peaking at `std_max` = 0.008 (98.33% / 98.67%).
- **PushT** (contact-heavy manipulation): clean peaks at `std_max` = 0.003 (89.67%), but robustness at px+goal 0.08 peaks at `std_max` = 0.006 (87.00%) — clean and robust optima dissociate.
- **Reacher** (continuous reaching): lies on a 0.002–0.006 plateau; clean point-best is at `std_max` = 0.006 (86.00%), while px+goal 0.08 point-best is at `std_max` = 0.002 (85.67%). Very low noise (0.001) is statistically indistinguishable from base.
- **Cube** (structured manipulation): noise sweep is weakest; no monotone trend on clean.

This exposes a fundamental tension: **global noise augmentation cannot distinguish "background visual redundancy that should be made invariant" from "control-relevant features that should retain resolution".**

1.3 Contributions

C1. A systematic quantification of visual-OOD fragility of JEPA + CEM control across four representative tasks. We sweep 4 tasks × 8 noise levels, and evaluate all 36 checkpoints under a unified 3-evaluation-seed × 100-trajectory protocol, for 300 trajectories per condition.

C2. The "invariance–resolution trade-off" concept and a five-layer diagnostic protocol that operationalises it. We define five complementary diagnostic layers (encoder shift, encoder geometry, predictor sensitivity, latent-noise response, task resolution) with 17+ concrete metrics, validated by Spearman correlations on the 4-task × $n = 9$ LeWM sweep under unified 3-seed × 100 evaluation, with partial correlations conditioned on training noise to separate residual ckpt-quality signal from the monotone sweep trend induced by `std_max`.

C3. A mechanistic account of why global noise augmentation has limited returns. The diagnostic layers show that on TwoRoom the gains coincide with desirable representation compression (effective rank 47.60 $\to$ 33.59) — a low-dimensional discrete task does not need high resolution — whereas PushT starts from a high-resolution, high-controllability representation (effective rank 76.42, `id_probe_r`^{2} 0.7739). Even the light representative PushT diagnostic checkpoint compresses rank substantially (76.42 $\to$ 42.85) with modest downward movement in task-resolution metrics (`transition_resolution_ratio_`l2 0.3015 $\to$ 0.2800; `id_probe_r`^{2} 0.7739 $\to$ 0.7500), so further compression cannot be assumed safe for contact-heavy tasks.

C4. An honest delineation of cross-ckpt diagnostics. The strongest cross-checkpoint diagnostic (`predictor_target_to_nn_cos_ratio_at_max_std`) carries a residual ckpt-quality signal beyond the sweep-level `std_max` effect (partial Spearman $\rho = -0.59$ on clean and -0.41 on px+goal 0.08 after conditioning on `std_max`, on the $n = 9$ LeWM PushT sweep with unified 3-seed $\times$ 100 eval). However, the apparent $\rho(\text{metric}, \text{OOD drop}) = -0.77$ is mediated by `std_max`: after partialling out `std_max`, that correlation collapses to +0.06. The metric is a model-selection tool after controlling for the sweep-level `std_max` trend, not an OOD oracle (Sections 4.5 and 5.3).

1.4 Organisation

Section 2 reviews related work; Section 3 defines the LeWM background, noise protocol, and diagnostic framework; Section 4 reports the experimental findings; Section 5 discusses mechanism and implications; Section 6 concludes.

2 Related Work

2.1 JEPA and latent world models

JEPA [12] predicts in latent space rather than reconstructing pixels. I-JEPA [1] predicts target representations from a masked context; V-JEPA [3, 2] extends the framework to video understanding and video-driven world modelling; LeWM [13] is the first end-to-end stable JEPA world model, using **SIGReg** (Sketch Isotropic Gaussian Regularizer) — random projections plus Epps–Pulley empirical-characteristic-function matching [7] — to prevent representation collapse without batch normalisation. LeWM is our baseline. The original LeWM paper reports a Violation-of-Expectation experiment showing the model is sensitive to physical perturbations (object teleportation) but not to visual perturbations (colour change). VoE however measures prediction error (surprise), not control success rate, and colour change differs from pixel-level Gaussian noise. We give the first picture, to our knowledge, of LeWM's control success rate under pixel-noise corruption.

2.2 Robustness studies of JEPA

N-JEPA [15] introduces diffusion-noise augmentation into I-JEPA, improving linear-probing robustness on ImageNet. VJEPA [10] tests a Noisy-TV distractor on synthetic 1D signals and reports JEPA retains $R^2 > 0.84$ under high noise. US-JEPA [16] tests Gaussian blur, contrast reduction, and speckle noise on medical ultrasound. None of these studies the pixel-noise robustness of a JEPA *world model* on *robotic control*. Furthermore, VJEPA's optimistic conclusion ($R^2 > 0.84$

in 1D) contrasts with our control-time observation (success rate $\rightarrow$ 4.67%), suggesting that the "natural robustness" of JEPA may be an artefact of evaluation modality.

2.3 World models and input augmentation

In RL world-model literature, DreamerV3 [8] and TD-MPC2 [9] rely on learned visual encoders and latent dynamics, which may provide some implicit tolerance to benign visual variation but do not by themselves settle sensor-noise robustness. ViGMO [14] tests Gaussian noise and blur on DMC tasks, finds "sensor noise is a fundamentally different distribution shift", and proposes a latent-consistency loss. ViGMO addresses model-based RL, not JEPA architectures; its conclusion is directionally aligned with ours, but our contribution adds mechanistic decomposition via the five-layer diagnostic, not just performance reporting.

2.4 The invariance–resolution tension

Tamkin et al. [18] argue, in the contrastive-learning context, that label-destroying augmentations can be useful — augmentations act as feature dropout rather than pure invariance inducers. Zhang and Ma [20] note that augmentation modules can impose invariances that must be matched to the task. These insights are well-established in contrastive learning for classification, but their manifestation in *latent predictive world models* — where the downstream task is planning rather than discrimination — has not been systematically studied. The present paper extends the discussion from classification to control with a quantitative framework.

2.5 Representation diagnostics and collapse analysis

Self-supervised learning broadly uses effective rank [17], condition number, and participation ratio to diagnose dimensional collapse [11]. Next-Latent Prediction [19] uses effective latent rank to assess world-model compactness. VICReg [4] provides an anti-collapse regularizer distinct from SIGReg. Individual metrics are not new; our contribution is to combine them into a coherent five-layer protocol with per-token noise sensitivity, cross-ckpt correlation, and a partial-correlation validation scheme conditioning on training noise.

3 Background and Diagnostic Framework

3.1 LeWorldModel baseline

LeWM [13] is an end-to-end JEPA world model trained with two terms only:

$$\mathcal{L}_{\text{LeWM}} \;=\; \mathcal{L}_{\text{pred}} \;+\; \lambda \cdot \mathcal{L}_{\text{SIGReg}}\,, \tag{1}$$

where $\mathcal{L}_{\text{pred}}$ is the latent-space MSE between predicted and target representations. SIGReg projects each latent onto M unit-norm random directions, computes the Epps–Pulley empirical-characteristic-function distance [7] between each projection and $\mathcal{N}(0,1)$, and aggregates with Gauss-window weights, preventing collapse without explicit BatchNorm. (The Cramér–Wold theorem motivates the construction: equality of high-dimensional distributions reduces to equality of all one-dimensional projections of their characteristic functions.) Inference uses the Cross-Entropy Method (CEM) for latent-space MPC.

3.2 Input-side noise augmentation

We add per-frame Gaussian noise to the LeWM input pipeline via `utils.py::AddNormalizedGaussianNoise` (Appendix A). Each frame is independently noised: Bernoulli(p) decides whether the frame is corrupted, and if so the standard deviation is drawn from Uniform(0, std_max). We fix $p = 1.0$ and sweep `std_max` $\in \{0.001, \ldots, 0.008\}$ (eight levels). Evaluation uses two noised conditions: pixels+goal 0.05 and pixels+goal 0.08 (Gaussian noise of the indicated std applied to both pixels and the goal image).

3.3 Five-layer diagnostic framework

Layer 1 — Encoder shift. Quantifies the magnitude and direction of latent displacement under input noise. Metrics: `noise_angle_deg`, `noise_l2`, `noise_to_nn_cos_ratio`, `noise_angle_slope`.

Layer 2 — Encoder geometry. Quantifies global latent-space structure. Metrics: `clean_nn_cos_dist`, `clean_effective_rank`, `cka_linear_at_max_std`.

Layer 3 — Predictor sensitivity. Quantifies how the predictor amplifies input noise. Metrics: `predictor_target_to_nn_cos_ratio_at_max_std` (single-step predictor target shift normalised by clean NN distance — the strongest cross-ckpt diagnostic we identify), `predictor_rollout_drift_T(T)`.

Layer 4 — Latent-noise response. Injects noise directly in the latent z (bypassing the encoder) to isolate predictor and cost contributions. Metrics: `latent_cost_surface_slope_z`, `latent_robust_radius_z`.

Layer 5 — Task resolution. Quantifies control-relevant information retained by the latent. Metrics: `transition_resolution_ratio`_cos, `transition_resolution_ratio`_l2, `id_probe_r`^{2} (a controllability proxy).

3.4 Cross-checkpoint validation protocol

To rule out spurious correlations we adopt two complementary analyses on the same LeWM PushT sweep:

LeWM $n = 9$ sweep with partial correlation. All 9 LeWM checkpoints per task (base + `std_max` $\in \{0.001, \ldots, 0.008\}$). Compute Spearman ρ and *partial* Spearman conditioned on `std_max`. The partialling step matters: many diagnostics correlate with control performance only because both quantities co-vary with the training noise level along the sweep. The relevant question is whether a diagnostic retains a residual ckpt-quality signal after removing the monotone sweep trend associated with `std_max`.

We report both the raw Spearman and the partial-on-`std_max` quantities. The raw ρ tells you "which checkpoint over the entire sweep behaves better"; the partial ρ tells you whether a diagnostic retains a residual ckpt-quality signal after removing the monotone trend associated with `std_max`. The two questions are distinct, and we will see in Section 4.5 that they have markedly different answers for the same metric. A larger cross-architecture protocol (varying the latent geometry of the world model itself) is left to follow-up work with external baselines.

4 Experiments

4.1 Setup

Tasks. PushT (2D pushing), TwoRoom (2D navigation), Reacher (2D arm), Cube (3D manipulation).

Baselines. LeWM-base (no noise) and LeWM+noise (8-level sweep).

Checkpoints. The 36 checkpoints in this paper correspond to one trained model per (task, `std_max`) configuration.

Evaluation seeds. Each checkpoint is evaluated with 3 evaluation seeds (42 / 43 / 44), with 100 trajectories per seed.

Evaluation protocol. All success rates in this paper — clean and noised, across all 36 checkpoints (4 tasks × {base, `std_max` 0.001–0.008}) — are computed under a single protocol: $n = 3$ seeds (42/43/44), `num_eval = 100` trajectories per seed (300 trajectories per condition per checkpoint). Every cell of Tables 1, 2, 3 is mean ± across-seed population std over $n = 3$, matching `assets/paper1_data/canonical_evals_20260517.json`. Raw per-seed metrics live at `<ckpt>/eval_results/<cond>_se`
the aggregated source-of-truth for downstream evaluation analysis is `assets/paper1_data/canonical_evals_2026051`
The released diagnostic source-of-truth for Figure 5, Table 5, Table 6, and Table 7 is `assets/paper1_data/canonical_`

Hardware. Single NVIDIA A100 (80 GB); training takes 2–4 h per task per configuration.

Figures. Six main figures are produced by `tools/paper1_figs.py`. Figures 1 to 6 are referenced in the relevant sections.

4.2 JEPA control fragility under visual OOD

Table 1 reports LeWM-base success rates under clean and noised evaluation (mean ± std across 3 seeds × 100 evaluations).

Table 1: LeWM-base under visual OOD (mean ± std; 3 seeds × 100 evaluations).

Task	clean	px+goal 0.05	px+goal 0.08	clean → 0.08 drop
TwoRoom	94.00 ± 3.56	61.33 ± 5.31	50.00 ± 1.41	**−44.00**
PushT	86.33 ± 2.36	12.00 ± 4.55	4.67 ± 2.05	**−81.67**
Reacher	58.67 ± 1.25	27.00 ± 5.10	15.00 ± 2.16	**−43.67**
Cube	66.67 ± 2.62	53.33 ± 3.30	46.33 ± 3.68	**−20.33**

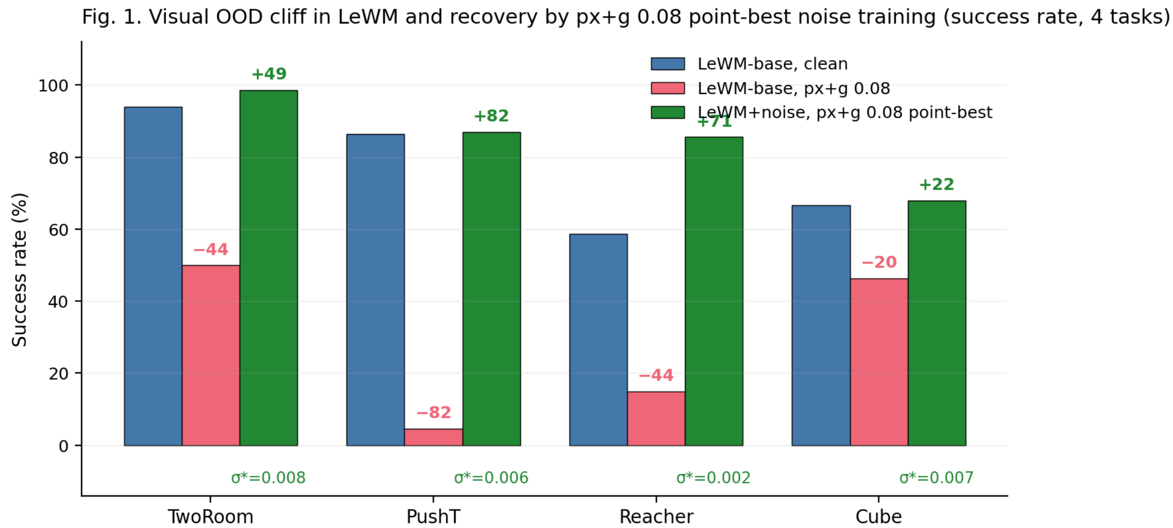

Figure 1: Visual OOD cliff in LeWM and recovery by noise training (success rate, 4 tasks). Blue: LeWM-base clean. Red: LeWM-base under px+g 0.08 noise. Green: the px+g 0.08 point-best LeWM+noise configuration. Annotations show base drop and recovery in points; σ^* marks each task's px+g 0.08 point-best `std_max`.

LeWM-base is strong on clean images (especially TwoRoom and PushT), but visual std $= 0.05$ applied to pixels and goal jointly already produces large drops on all tasks: PushT loses 74+ pts (down to near-random 4.67% at std $= 0.08$), TwoRoom 30+ pts, Reacher 30+ pts, Cube ~ 13 pts (at std $= 0.05$). **This is not a marginal phenomenon**: a JEPA + CEM world model without noise-aware training has essentially no resistance to visual corruption. The drop pattern across tasks is informative: Cube degrades least (-20.33 pt at std $= 0.08$) — structured manipulation has some natural robustness — while PushT degrades most catastrophically (-81.67 pt), confirming that contact-heavy continuous control is most sensitive to visual precision.

4.3 Noise augmentation closes the gap — at the cost of task-specific tuning

Tables 2 and 3 report the complete 8-level sweep across tasks. All cells are success rate $\pm$ across-seed std (in pts), aggregated under the unified $n = 3$ seeds (42/43/44) $\times$ 100 trajectories protocol — 300 trajectories per cell.

Table 2: LeWM+noise sweep, **clean** success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	94.00 ± 3.56	86.33 ± 2.36	58.67 ± 1.25	66.67 ± 2.62
0.001	93.67 ± 3.30	88.00 ± 3.74	61.67 ± 2.49	69.33 ± 0.47
0.002	95.00 ± 2.83	88.33 ± 2.62	85.67 ± 2.49	60.00 ± 1.63
0.003	96.33 ± 3.30	$89.67 \pm 1.70^{\dagger}$	78.67 ± 1.25	65.00 ± 1.63
0.004	96.33 ± 2.05	89.33 ± 2.05	84.00 ± 2.94	69.00 ± 3.74
0.005	96.00 ± 2.83	80.67 ± 4.78	70.00 ± 2.16	59.33 ± 0.94
0.006	96.67 ± 2.05	89.33 ± 2.05	$86.00 \pm 2.94^{\dagger}$	66.67 ± 2.05
0.007	96.00 ± 1.63	85.67 ± 3.09	83.67 ± 3.30	67.67 ± 0.94
0.008	$98.33 \pm 0.47^{\dagger}$	88.33 ± 2.87	84.00 ± 0.82	62.33 ± 1.25

Table 3: LeWM+noise sweep, **px+goal 0.08** success rate (%).

`std_max`	TwoRoom	PushT	Reacher	Cube
0 (base)	50.00 ± 1.41	4.67 ± 2.05	15.00 ± 2.16	46.33 ± 3.68
0.001	87.67 ± 1.89	43.33 ± 3.09	46.00 ± 1.63	51.33 ± 5.79
0.002	93.33 ± 0.94	71.33 ± 3.68	$85.67 \pm 1.70^{\ddagger}$	60.67 ± 0.47
0.003	94.67 ± 2.87	83.00 ± 3.74	73.67 ± 0.47	67.33 ± 1.89
0.004	95.00 ± 2.45	81.33 ± 2.87	80.00 ± 1.41	67.00 ± 3.56
0.005	95.67 ± 2.36	75.00 ± 6.48	68.00 ± 3.56	59.67 ± 2.05
0.006	96.67 ± 2.49	$87.00 \pm 3.74^{\ddagger}$	84.67 ± 4.03	65.00 ± 2.94
0.007	96.33 ± 2.05	82.33 ± 4.64	81.33 ± 1.25	$68.00 \pm 1.41^{\ddagger}$
0.008	$98.67 \pm 0.94^{\ddagger}$	85.33 ± 2.62	83.00 ± 4.32	60.33 ± 0.94

Reading guidance. † marks the clean point-best in that task column; ‡ marks the px+goal 0.08 point-best. Table 4 and Figure 4 use a separate term, *representative diagnostic checkpoint*, for the ckpt on which the full diagnostic suite was executed. Because many neighbouring settings differ by ≤ 3 pts and overlap in across-seed std, we interpret optima as plateaus unless the gap is clearly larger than the seed-level variability.

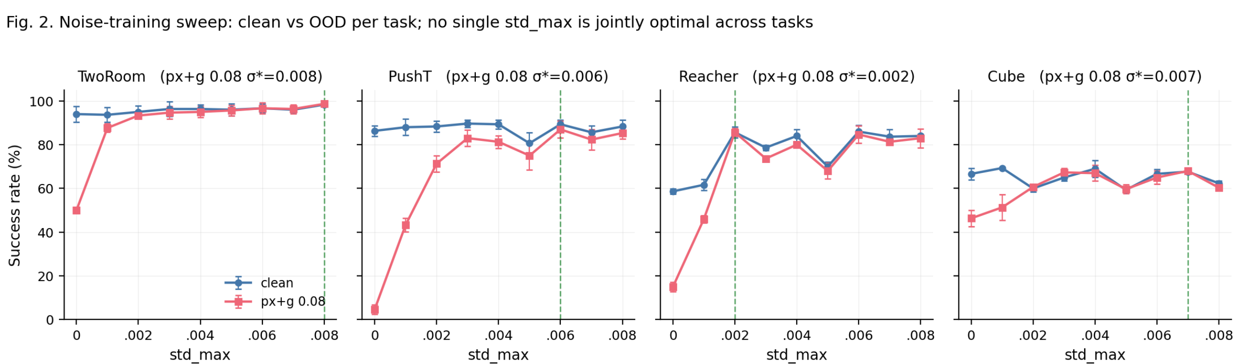

Figure 2: Noise-training sweep: clean vs OOD per task. Dashed vertical lines mark each task's px+g 0.08 point-best σ^*. No single `std_max` is jointly optimal across tasks.

Three observations follow.

(1) No single `std_max` is jointly optimal; clean and robust optima can dissociate within a task. TwoRoom peaks globally at `std_max` = 0.008 (98.33 / 98.67); clean rises monotonically with noise. PushT peaks on clean at `std_max` = 0.003 but on robustness at `std_max` = 0.006 (87.00 vs. 0.002's 71.33; +15.67 pt). Reacher lies on a 0.002–0.006 plateau: clean point-best is at `std_max` = 0.006 (86.00), while px+g 0.08 point-best is at `std_max` = 0.002 (85.67). At `std_max` = 0.001 clean is 61.67 vs base 58.67 — within across-seed std ($\sim$ 2.5 pts), so the data support *"low noise is statistically equivalent to base"*, not "low noise hurts". Cube responds least: clean is non-monotonic, and px+g 0.08 improves only in the 0.003–0.007 range.

(2) Per-task tuning is necessary, not optional. Optimal `std_max` varies substantially: TwoRoom clean/OOD point-best 0.008, PushT clean point-best 0.003 / px+g 0.08 point-best 0.006, Reacher clean point-best 0.006 / px+g 0.08 point-best 0.002, Cube px+g 0.08 point-best 0.007 with a shallow clean plateau around 0.001 / 0.004 / 0.007. Global input-side noise is the strongest "global" form of invariance pressure, but closing the OOD gap requires per-task tuning cost.

(3) Tasks form a clear sensitivity ordering at the extremes. PushT is clearly most sensitive (-81.67 base drop), Cube least sensitive (-20.33), while TwoRoom (-44.00) and Reacher (-43.67) are effectively tied around a 44-pt drop. Recovery does not scale with sensitivity: TwoRoom recovers most fully ($+48.67$ pt at `std_max` $= 0.008$), Cube least ($+21.67$ pt). Input-side global noise is most effective on visually redundant tasks and gives diminishing returns on structured manipulation.

Plotting the same sweep as a (clean, OOD) trajectory per task (Figure 3) makes the trade-off visually explicit: each task's sweep curve moves from base far below the $y = x$ diagonal toward the upper-right, with per-task curvature determined by how much OOD gain a task can buy from a given drop in clean.

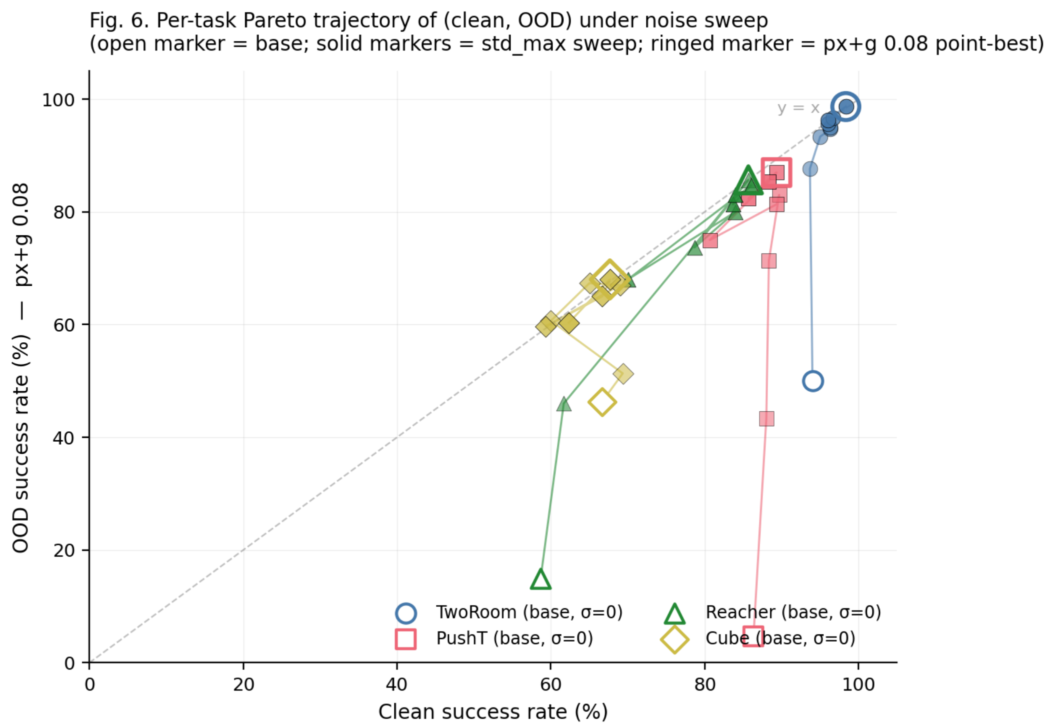

Figure 3: Per-task Pareto trajectory of (clean, OOD) under noise sweep. Open marker = base; solid markers = `std_max` sweep (colour-by-`std_max`); ringed marker = px+g 0.08 point-best. Dashed line $y = x$.

4.4 Diagnostic analysis: why global noise is not a silver bullet

Table 4 compares core diagnostic metrics on LeWM-base versus the per-task representative noise-trained diagnostic checkpoint.

Table 4: Representation diagnostics: LeWM-base vs. a representative noise-trained diagnostic checkpoint per task. The σ pick per task (0.008 / 0.002 / 0.006 / 0.001) is the ckpt on which the diagnostic suite was originally executed. Under the unified 3-seed $\times$ 100 eval protocol the PushT clean point-best shifts to `std_max` = 0.003 (within ± 2 pt of `std_max` = 0.002); we keep the diagnostics on the `std_max` = 0.002 checkpoint because the compression-vs.-resolution pattern this table illustrates is robust within this neighbourhood.

Metric	TwoRoom base	TwoRoom noise (0.008)	PushT base	PushT noise (0.002)	Reacher base	Reacher noise (0.006)	Cube base
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051	0.0633	0.0676	0.1856
`clean_effective_rank`	47.60	33.59	76.42	42.85	61.04	65.92	73.25
`transition_resolution_ratio__l2`	0.7216	0.6055	0.3015	0.2800	0.3704	0.3791	0.4847
`transition_resolution_ratio__cos`	0.5538	0.3780	0.0868	0.0800	0.1351	0.1399	0.2347
`id_probe_r2`	0.2889	−0.0573	0.7739	0.7500	0.1621	0.1729	0.6657
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.1200	0.2518	0.2585	0.2364
`predictor_rollout_T8_l2`	18.62	17.90	18.65	16.50	15.17	0.44	20.20

Notes. (i) `transition_resolution_ratio__l2` and `__cos` values for TwoRoom are corrected to the values directly stored in `geometry_summary.json` / `task_resolution.json`; (ii) Reacher and Cube representative diagnostics are taken from the corresponding ckpt's `eval_results/diagnostics/{geometry,task,predictor` (max-std = 0.1, history-only noise); (iii) Cube base `predictor_rollout_T8_l2` = 20.20 and Reacher base = 15.17 are comparable, but the recovery effect of noise training on long-horizon rollout drift is highly task-dependent: Reacher representative = 0.44 (35$\times$ reduction); Cube representative = 19.25 (almost unchanged).

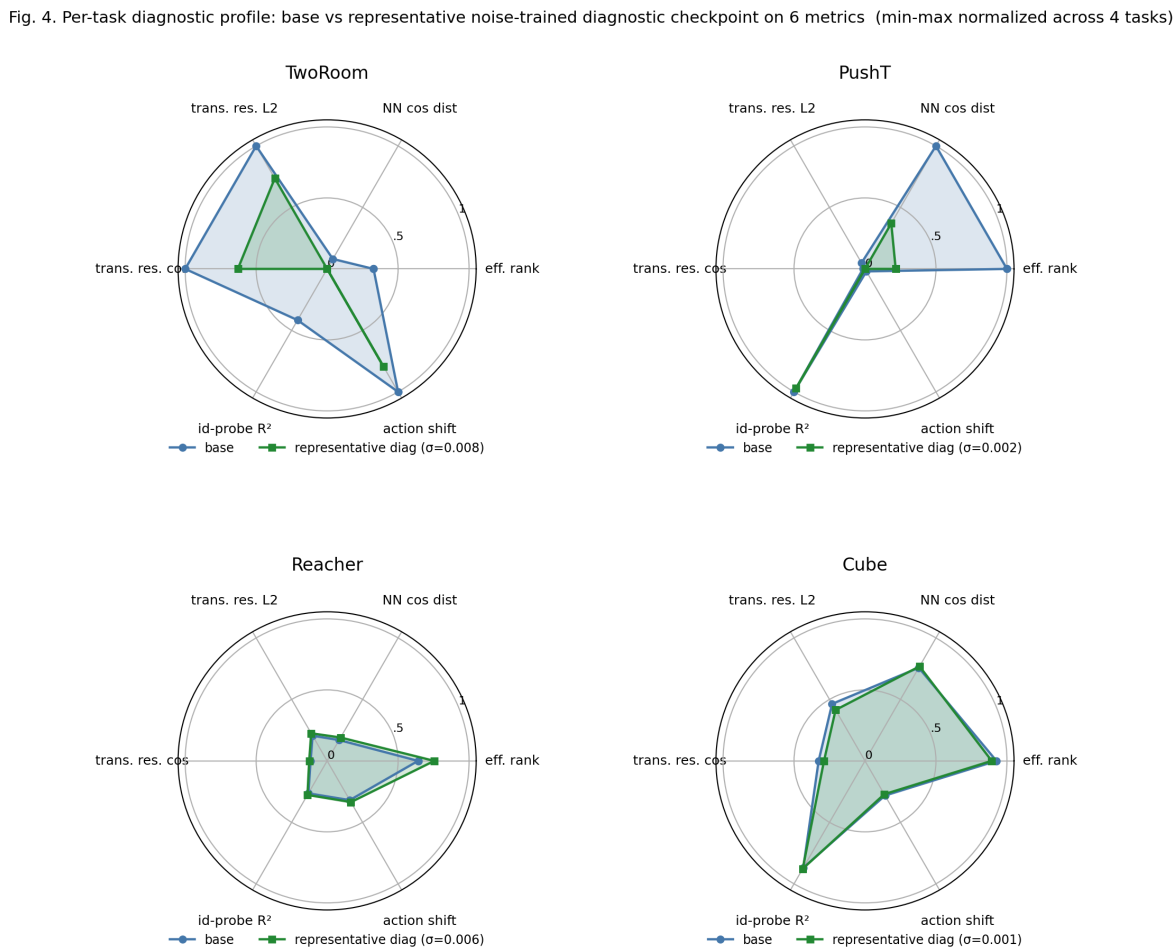

Figure 4: Per-task diagnostic radar: base vs representative noise-trained diagnostic checkpoint on 6 metrics, min-max normalised across all 4 tasks.

Mechanistic reading.

- **TwoRoom.** Low-dimensional, discrete, visually redundant. Compressing the representation (effective rank 47.6 $\rightarrow$ 33.6) is acceptable and even beneficial: a more compact latent space is easier to plan in.
- **PushT.** Continuous contact requires fine-grained pose resolution. Even at the representative diagnostic checkpoint (`std_max` $= 0.002$), `transition_resolution_ratio_l2` already trends slightly downward. At heavier noise (e.g. 0.006) the metric would drop further, erasing contact-transition keyframes.
- **Predictor-rollout caveat.** A drop in `predictor_rollout_T8_l2` is not unambiguously good news. It can also mean the latent has become more *predictable* without being more *controllable*: predictor stability bought by sacrificing resolution.

4.5 Cross-checkpoint correlation analysis

Table 5 reports task-specific cross-ckpt correlations on the LeWM $n = 9$ sweep using the unified 3-seed $\times$ 100 evaluation.

We analyse two single-value-per-checkpoint diagnostic metrics that have full coverage on all 9 checkpoints per task: `predictor_target_to_nn_cos_ratio_at_max_std` (the "fragility metric" —

single-step predictor target shift normalised by NN distance, at the diagnostic's max-std injection) and `predictor_rollout_T8_l2_at_max_std` (the corresponding multi-step drift). Both are released in `assets/paper1_data/canonical_diagnostics_20260517.json`, derived from each checkpoint's `eval_results/diagnostics/predictor_sensitivity.json`, and are pure ckpt-level quantities (independent of the evaluation protocol).

Table 5: LeWM $n = 9$ sweep: per-task Pearson r / Spearman ρ vs OOD drop (clean − px+g 0.08). Eval values come from the unified 3-seed × 100 protocol.

Metric	TwoRoom (r/ρ)	PushT (r/ρ)	Reacher (r/ρ)	Cube (r/ρ)
`predictor_target_to_nn_cos_ratio_at_max_std`	+0.96/+0.72	**−0.54**/**−0.77**	+0.97/+0.35	+0.36/+0.21
`predictor_rollout_T8_l2_at_max_std`	+0.89/+1.00	+**0.99**/+**0.88**	+**0.99**/+**0.87**	+0.92/+0.54

Reading. Unconditional correlations are large because both diagnostic metrics and the OOD drop are driven by the same upstream variable — `std_max`. The relevant test is partial correlation conditioned on `std_max` (Table 6).

Table 6: Partial Spearman ρ(metric, OOD drop | `std_max`), $n = 9$ per task.

Metric	TwoRoom	PushT	Reacher	Cube
`predictor_target_to_nn_cos_ratio_at_max_std`	— (rank ties)	+0.06	−0.12	+0.14
`predictor_rollout_T8_l2_at_max_std`	— (rank ties)	−0.03	+**0.79**	+0.50

The TwoRoom partial estimates are undefined because saturating success rates produce rank ties that make residualisation singular at $n = 9$. PushT, Reacher and Cube give clean readings: after removing the monotone sweep trend associated with `std_max`, the **fragility metric carries essentially no information about the size of the OOD drop** on any task; the **multi-step predictor drift** still carries signal on Reacher (+0.79) and weak signal on Cube (+0.50). The Reacher partial ρ is the only non-trivial residual correlation in the entire matrix.

Table 7: Spearman ρ (unconditional) and partial Spearman ρ conditioned on `std_max`, for the fragility metric on the PushT $n = 9$ LeWM sweep under unified 3-seed × 100 eval.

Quantity	Spearman ρ
ρ(`std_max`, metric)	+0.83
ρ(`std_max`, clean)	0.00
ρ(`std_max`, px+goal 0.08)	+0.82
ρ(`std_max`, OOD drop)	−0.93
ρ(metric, clean) unconditional	−0.33
ρ(metric, clean) | `std_max` partial	**−0.59**
ρ(metric, px+g 0.08) unconditional	+0.55
ρ(metric, px+g 0.08) | `std_max` partial	**−0.41**
ρ(metric, OOD drop) unconditional	−0.77
ρ(metric, OOD drop) | `std_max` partial	+0.06

PushT $n = 9$ detailed view (fragility metric). The honest interpretation:

1. **After removing the monotone trend associated with `std_max`, the metric still carries a meaningful residual ckpt-quality signal.** Partial ρ on clean is -0.59 and on px+goal 0.08 is -0.41 — both negative, both meaningful at this sample size. On the $n = 9$ PushT sweep, lower fragility ratio aligns with better clean *and* better noisy success beyond what the sweep-level `std_max` trend already explains.
2. **The metric does NOT predict the clean–OOD gap.** The unconditional $\rho(\text{metric}, \text{drop}) = -0.77$ looks impressive, but partialling out `std_max` collapses it to $+0.06$ (sign-flipped, near zero). The drop's strong correlation with the metric is a mediated effect: `std_max` drives both the metric ($\rho = +0.83$) and the drop ($\rho = -0.93$). After the `std_max` trend is removed, the metric tells you nothing new about how much the *gap* between clean and noisy performance will be.
3. **Note the unconditional-vs-partial sign reversal on clean.** $\rho(\text{metric}, \text{clean})$ is only -0.33 unconditionally — clean success rate is roughly flat across the PushT sweep under the 3-seed protocol (range 80.67–89.67), so `std_max` barely moves clean. The partial correlation *strengthens* to -0.59 once the sweep-level `std_max` trend is removed, exactly because the residual signal is what the metric tracks.
4. **Practical reading.** The toolkit is a model-selection tool that ranks checkpoints after controlling for the sweep-level `std_max` effect. It is *not* a substitute for actually training with noise when the OOD gap is the quantity of interest.

4.5.1 What this diagnostic actually predicts: clean vs OOD

The partial-correlation analysis above settles the interpretation:

- The metric is a **residual ckpt-quality signal beyond the sweep-level `std_max` effect** — lower fragility ratio means better control on *both* clean and noisy evaluation (partial $\rho = -0.59$ / -0.41 on PushT).
- The metric **does not isolate noise-robustness** — its apparent correlation with the gap between clean and noisy success is fully mediated by `std_max` (partial $\rho = +0.06$).

The PushT scatter (Figure 5) makes both halves visible. Panel (a) plots metric vs clean (unconditional Spearman $\rho = -0.33$; the residual trend after conditioning on `std_max` is what the partial correlation captures). Panel (b) plots metric vs OOD drop (unconditional $\rho = -0.77$), but the colour-bar (encoding `std_max`) reveals the structure: low-`std_max` ckpts (light blue) live at high drop, high-`std_max` ckpts (dark blue) live at low drop, and the metric tracks `std_max` itself. After the `std_max` trend is removed, the metric does not separate small-drop from large-drop ckpts.

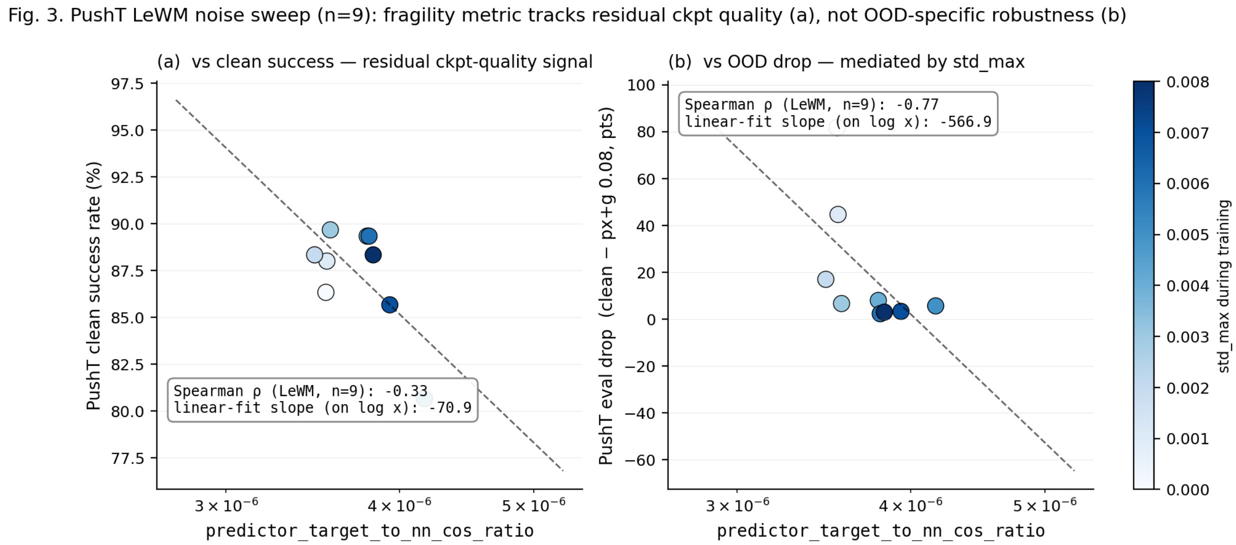

Figure 5: PushT $n = 9$ LeWM sweep: fragility metric is a ckpt-quality predictor (a), the apparent OOD-drop correlation in (b) is mediated by `std_max` (encoded by the colour bar).

4.6 Mechanism attribution: where in the pipeline does noise cause failure?

Section 4.4 reports *what* is compressed and Section 4.5 reports *which diagnostics* correlate with eval across checkpoints, but neither answers **"is the failure in the encoder, the predictor, or the cost surface?"** We address this with two complementary experiments.

4.6.1 Auxiliary cost-swap sanity check: cost alone is unlikely to explain the collapse

We ran a one-off eval-only cost swap on a single TwoRoom checkpoint outside the 36-checkpoint canonical evaluation table; full details are reported in Appendix E. Switching the CEM cost from cosine/normalized to mse/raw changes px+goal 0.03 success only from 36.0 to 42.0, still far below a separate clean reference of 69.7. As a sanity check, this suggests that the cost function alone is unlikely to explain the OOD collapse.

4.6.2 Latent-noise probing: encoder is the principal bottleneck

Directly injecting noise into the latent z (skipping the encoder) decouples encoder contributions from predictor + cost. Comparing diagnostic signals across two injection points (pixels vs. latent z, both history-only noise at max std):

- **PushT.** The multi-step input-space metric `predictor_rollout_T8_l2_at_max_std` has unconditional $\rho = +0.88$ vs OOD drop (Table 5), driven primarily by `std_max`; after removing the monotone `std_max` trend, the partial ρ collapses to -0.03. The single-step `predictor_target_to_nn_cos_ratio_at_max_` (unconditional $\rho = -0.77$; partial $+0.06$ after removing the same trend) tells the same story. Both encoder–predictor signals are mediated by training noise; once that sweep-level effect is accounted for, neither isolates OOD drop on PushT.
- **Reacher.** `predictor_rollout_T8_l2_at_max_std` has unconditional $\rho = +0.87$ *and* partial $\rho = +\mathbf{0.79}$ vs OOD drop — the only non-trivial residual correlation in the matrix. On Reacher, multi-step predictor drift carries genuine OOD-drop information beyond `std_max`.

- **Cube.** `predictor_rollout_T8_l2_at_max_std` has unconditional $\rho = +0.54$ and partial $\rho = +0.50$ — a moderate residual signal. Encoder–predictor drift partially explains Cube's small but non-zero OOD sensitivity.
- **TwoRoom.** Partial correlations are undefined because clean / drop are saturated across the sweep (rank ties). Unconditional ρ still indicates strong encoder–predictor sensitivity to `std_max`.

Table 8 summarises the three-layer attribution.

Table 8: Three-layer attribution by task (LeWM $n = 9$ sweep, unified 3-seed $\times$ 100 eval).

Task	Primary cause	Residual after removing `std_max` trend
TwoRoom	encoder dominant (rank-saturated)	n/a
PushT	encoder + single-step predictor (both mediated by `std_max`)	no residual metric isolates OOD drop
Reacher	encoder + multi-step rollout	multi-step drift residual signal (partial $\rho = +0$
Cube	encoder	moderate residual on multi-step drift (partial

The common primary cause across the four tasks is **encoder shift transduced by the predictor**. The auxiliary cost-swap sanity check in Section 4.6.1 suggests that the cost function alone is unlikely to explain the collapse, but we do not treat that single-checkpoint ablation as a task-wide quantitative attribution. This is also the root reason Layer-5 task-resolution metrics carry strong signal in Section 4.4: once the encoder's latent-neighbourhood structure is corrupted beyond the NN distance scale, downstream predictor and planner are already operating on the wrong neighbourhood.

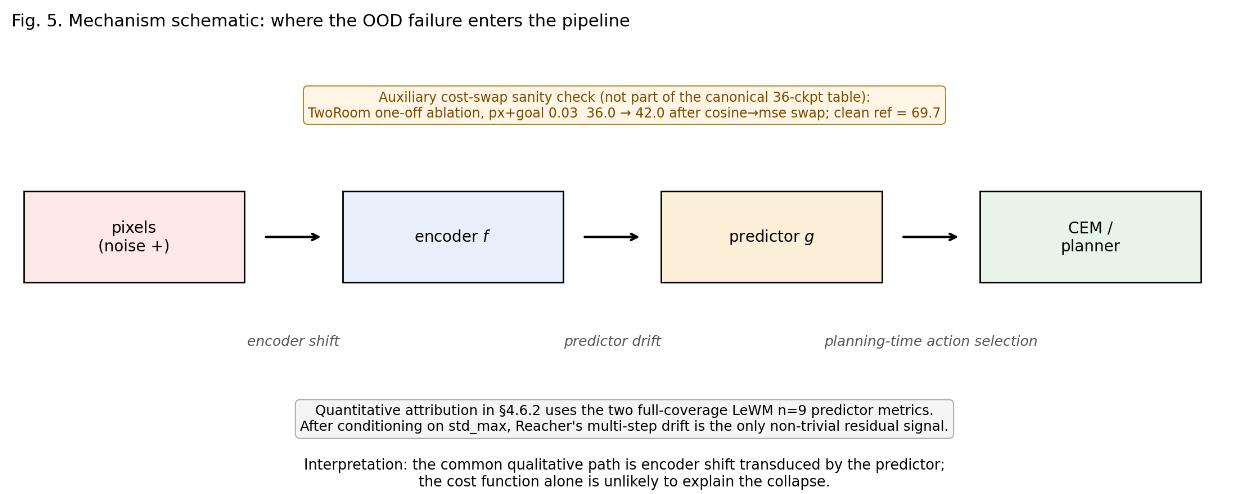

Figure 6: Mechanism schematic. Pixels perturb the encoder, the predictor transduces that shift into planning-relevant latent drift, and CEM acts on the resulting latent geometry. Quantitative attribution in §4.6.2 uses the two full-coverage LeWM $n = 9$ predictor metrics; after conditioning on `std_max`, Reacher's multi-step drift is the only non-trivial residual signal.

5 Discussion

5.1 Rethinking the "JEPA invariance" narrative

The data challenge an implicit community assumption: latent prediction alone is not sufficient to confer invariance to visual noise. The invariance a JEPA encoder acquires is *in-distribution* *invariance* — invariance to visual patterns present in the training distribution. When test-time inputs carry high-frequency pixel noise not seen during training, the topology of the latent space breaks down, the predictor outputs incorrect future states, and the planner fails.

This stands in contrast to VJEPA [10], which reports $R^2 > 0.84$ under a Noisy-TV distractor — but that result is on 1D synthetic signals evaluated by linear probe. **Control-task evaluation (success rate) is a strictly stricter standard than linear-probe R^2**: linear probes only need enough information for a linear classifier to extract, whereas control tasks demand a representation that supports precise latent-space planning and action optimisation.

5.2 The invariance–resolution trade-off — manifestation and scope

The trade-off has the following four-task signature in this paper:

- **TwoRoom.** Background wall colour / texture is pure visual redundancy; discarding it does not impair navigation.
- **PushT.** The pixel changes during T-block / arm contact carry critical force and pose information; discarding them blinds the planner.
- **Reacher.** Low-dimensional continuous control; visual encoding of joint angle requires moderate resolution.
- **Cube.** Object pose and grasp-point spatial relations need moderate resolution, but Cube itself is largely insensitive to global noise (action sequence is structured; visual–action coupling is predictable).

Task-specificity explains why there is no single optimal noise level.

Scope. Whether the trade-off generalises to other latent world-model architectures is an open question we do not address. Reconstruction-based world models such as DreamerV3 [8] explicitly model observations, while decoder-free latent MPC systems such as TD-MPC2 [9] may exhibit different compression dynamics. ViGMO [14] observes related task-specific noise sensitivity on DMC. EMA-target JEPA (I-JEPA / V-JEPA lineage) and variational JEPA may exhibit different dynamics. This paper's scope is LeWM + CEM; universalising the trade-off exceeds the present evidence.

5.3 When the diagnostic toolkit works — and when it does not

Three boundaries should be stated up-front so practitioners know when to use the toolkit and when to fall back to direct evaluation.

Boundary 1: the toolkit ranks residual checkpoint quality, not OOD-specific robustness. Section 4.5.1 shows that on PushT, after conditioning on `std_max`, the strongest cross-ckpt diagnostic (`predictor_target_to_nn_cos_ratio_at_max_std`) has partial Spearman $\rho = -0.59$ with clean success and $\rho = -0.41$ with px+goal 0.08 success. It does *not* isolate OOD-specific robustness: the partial correlation with the clean-to-OOD drop is only $+0.06$. Use the toolkit to pick the strongest checkpoint after removing the sweep-level `std_max` trend; do not use it as a substitute for an actual OOD eval if your downstream question is robustness.

Boundary 2: tasks with weak per-state controllability variance fall outside the toolkit's strict regime. Reacher and TwoRoom do not produce diagnostic-vs-eval Spearman ρ that survives the partial-correlation criterion. Both tasks lack enough residual spread after accounting for `std_max` to distinguish "good" from "bad" ckpts via a label-free metric. The toolkit can describe what these models have compressed (Table 4) but cannot predict relative checkpoint quality.

Boundary 3: cross-ckpt diagnostics cannot recover what training did not provide. The largest determinant of OOD drop is whether the model saw noise during training — not which fragility metric it happens to score on. This is the structural reason $\rho(\text{metric}, \text{drop})$ is weak even when $\rho(\text{metric}, \text{clean})$ is strong: noise vs no-noise training puts each ckpt on a completely different (clean, OOD) curve (cf. Figure 3), and no static cross-ckpt diagnostic can substitute for that training-time choice.

The toolkit therefore has a precise scope: it is a *clean-evaluation auxiliary* that lets you select among ckpts on tasks with strong per-state controllability variance (PushT, Cube), assuming the training protocol is already fixed. It is not an OOD prediction oracle.

5.4 Practical guidance: how to choose `std_max` on a new task

Our sweep suggests a simple operational recipe:

1. Inspect the clean baseline's `predictor_target_to_nn_cos_ratio_at_max_std` as a screening diagnostic, not as an OOD oracle. Very small values (PushT / Cube regime) indicate that local encoder–predictor shift is small relative to the clean NN scale, but they do not by themselves rank OOD sensitivity.
2. Check `clean_effective_rank`, `transition_resolution_ratio`, and task semantics together. PushT combines high rank and high controllability (`id_probe_r`$^2 = 0.7739$), so noise should be swept with clean performance as a guardrail. TwoRoom is visually redundant and retains high transition separability (`transition_resolution_ratio`_l2 $= 0.7216$), so a wider sweep up to 0.008+ is reasonable.
3. Use *two endpoints* in evaluation (clean and max-noise). Checking only one misses one of the optima: PushT's clean optimum at 0.003 vs robustness optimum at 0.006 is the clearest example.
4. Under compute budget, start with a coarse 4-level sweep ($\{0.001, 0.003, 0.005, 0.007\}$), then refine around the best clean/OOD candidates. The coarse grid is a screening step, not a guarantee of locating the exact point-best `std_max`.

5.5 Limitations and future directions

Limitation 1 — Single backbone family. We validate on LeWM. Other JEPA variants (EMA-target I-JEPA / V-JEPA lineage; variational JEPA) may exhibit different noise responses.

Limitation 2 — Gaussian pixel noise only. Real-world visual corruption includes motion blur, contrast variation, occlusion, and lighting change; whether the trade-off transfers is open.

Limitation 3 — Diagnostic framework is empirical, not theoretical. Reacher and TwoRoom fail the partial-correlation criterion for *all* metrics, exposing where the empirical framework breaks down.

Limitation 4 — Automated transition reweighting is not a substitute for noise training. As a sanity check we also tested a scale-preserving heteroscedastic-NLL formulation in which a learned per-transition σ-head down-weights high-error transitions. The result is informative as a negative finding: TwoRoom clean reaches 99.67% but high-noise robustness lags noise training;

PushT clean *collapses* from 86.33% to 13.33% because contact-control transitions have high prediction error and are exactly the transitions σ-weighting would discard. Full data are reported in Appendix D. The lesson — "hard $\neq$ unimportant" in contact control — connects this paper's trade-off to the broader question of per-token controllers, which we leave to follow-up work.

Future direction 1 — Per-token adaptive consistency. Sections 4.5 and 4.6 identify the strongest signal `predictor_target_to_nn_cos_ratio_at_max_std`, a ckpt-level scalar. Whether its per-token variant can serve as a per-token consistency controller is an open methodological question, under investigation in follow-up work.

Future direction 2 — Cross-architecture replication. Running the sweep and diagnostic protocol on external world-model baselines would reveal whether the trade-off is JEPA-specific or shared by broader latent world-model families.

Future direction 3 — Theoretical grounding. Reformulating the trade-off in an information-bottleneck / rate-distortion language is an attractive direction; we did not pursue it because the empirical phenomenology may not yet be stable enough for formal modelling.

6 Conclusion

This paper provides a systematic diagnostic study of the visual-OOD control robustness of JEPA + CEM world models, using LeWorldModel as the representative system. Three findings:

1. Visual OOD collapse in JEPA + CEM control. Without noise-aware training, LeWM collapses under pixel noise; latent prediction alone does not confer visual robustness.
2. Global noise augmentation has a boundary. It effectively closes the OOD gap but has no globally optimal level; task differences are large, and within a single task clean and robustness optima can dissociate.
3. A five-layer diagnostic protocol reveals the underlying mechanism. Noise-augmentation gains arise from representation compression, but excessive compression destroys the resolution required for control — this is the *invariance–resolution trade-off*. When used cross-ckpt, the strongest single diagnostic (`predictor_target_to_nn_cos_ratio_at_max_std`) predicts clean control quality rather than OOD-specific robustness; we delineate this scope honestly.

This paper does not propose a new training algorithm. Its contribution is a systematic body of empirical evidence, a reusable diagnostic toolkit, and an explicit delineation of what cross-ckpt diagnostics can and cannot predict.

Acknowledgements

The complete code and data are available at `https://github.com/qun-team/wm_exp`. We thank the LeWM authors for releasing the baseline framework on which this study builds.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video (v-jepa). *Transactions on Machine Learning Research / arXiv:2404.08471*, 2024.

[4] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[5] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.

[6] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of NeurIPS*, 2020.

[7] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[8] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[9] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[10] Yongchao Huang. Vjepa: Variational joint embedding predictive architectures as probabilistic world models. *arXiv preprint arXiv:2601.14354*, 2026.

[11] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[12] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[13] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[14] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[15] Yuping Qiu, Rui Zhu, and Ying-cong Chen. Improving joint embedding predictive architecture with diffusion noise (n-jepa). *arXiv preprint arXiv:2507.15216*, 2025.

[16] Ashwath Radhachandran, Vedrana Ivezić, Shreeram Athreya, Ronit Anilkumar, Corey W. Arnold, and William Speier. Us-jepa: A joint embedding predictive architecture for medical ultrasound. *arXiv preprint arXiv:2602.19322*, 2026.

[17] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *Proceedings of the European Signal Processing Conference (EUSIPCO)*, 2007.

[18] Alex Tamkin, Margalit Glasgow, Xiluo He, and Noah Goodman. Feature dropout: Revisiting the role of augmentations in contrastive learning. In *Proceedings of NeurIPS*, 2023.

[19] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S. Hu, Pratyusha Sharma, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*, 2025.

[20] Junbo Zhang and Kaisheng Ma. Rethinking the augmentation module in contrastive learning: Learning hierarchical augmentation invariance with expanded views. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16650–16659, 2022.

A Experimental details

A.1 Environments

- **PushT.** 2D continuous pushing; 20,000 expert episodes; $\sim$196 steps/episode; action dim 2 (direction + magnitude); 224×224 RGB.
- **TwoRoom.** 2D continuous navigation; 10,000 episodes; $\sim$92 steps; action dim 2; 224×224 RGB.
- **Reacher.** 2D arm control (DeepMind Control Suite); 10,000 episodes; 200 steps; action dim 2.
- **Cube.** 3D cube manipulation (OGBench); 10,000 episodes; 200 steps; action dim 7.

A.2 Noise augmentation implementation

The actual implementation is `utils.py::AddNormalizedGaussianNoise`. Key points: (i) noise is added on ImageNet-normalized tensors and must be divided by the per-channel std to retain "pixel-space std" semantics; (ii) sampling is per-frame independent over the leading frame dims, not per-batch; (iii) all sweeps in this paper fix `noise_prob` = 1.0 and `std_min` = 0 and vary only `std_max`.

```python
class AddNormalizedGaussianNoise:
    """Per-frame independent. Each frame draws Bernoulli(noise_prob) then
    std ~ Uniform(std_min, std_max). Pixel-space std is converted to
    normalized space by dividing by per-channel std before adding."""
    def __init__(self, std_min, std_max, noise_prob=1.0):
        self.std_low, self.std_high, self.noise_prob = std_min, std_max, noise_prob
        self.channel_std = torch.as_tensor(IMAGENET_STD) # (C,)

    def __call__(self, x):
        if self.std_high <= 0 or self.noise_prob <= 0:
            return x
        leading = x.shape[:-3]
        stds = torch.empty(leading, device=x.device, dtype=x.dtype).uniform_(
            self.std_low, self.std_high)
        if self.noise_prob < 1.0:
            mask = (torch.rand(leading, device=x.device) < self.noise_prob).to(x.dtype)
```

```
        stds = stds * mask
    per_frame_scale = stds.view(*leading, 1, 1, 1)
    channel_factor = (1.0 / self.channel_std.to(x.device, x.dtype)).view(
        *([1] * len(leading)), -1, 1, 1)
    scale = per_frame_scale * channel_factor # pixel-space std -> normalized
    return x + torch.randn_like(x) * scale
```

A.3 Evaluation protocol

Clean evaluation uses no noise, 100 trajectories per seed $\times$ 3 seeds (42/43/44), totalling 300 trajectories per condition. Noisy evaluation adds Gaussian noise of std $\in \{0.05, 0.08\}$ to both pixels and the goal image under the same 3-seed protocol. Success criterion is task-specific (PushT: T-block pose match; TwoRoom: target region; Reacher: joint-angle match; Cube: cube position match).

A.4 Compute

All experiments run on a single NVIDIA A100 (80 GB). Training: ~2 h/task for LeWM-base, ~2.5 h/task/configuration for LeWM+noise. Diagnostic analysis: ~30 min/task/checkpoint.

A.5 Main-figure rendering

The six main figures are produced by `tools/paper1_figs.py`. Run `python -m tools.paper1_figs -out-dir assets/paper1_figs`. The script reads `assets/paper1_data/canonical_evals_20260517.json` together with `assets/paper1_data/canonical_diagnostics_20260517.json`; no checkpoint or model loading is required.

B Full diagnostic profile of LeWM-base on four tasks

Table 9 summarises every core diagnostic on the four LeWM-base checkpoints. Data are taken from each checkpoint's `eval_results/diagnostics/{geometry,task,predictor,latent_noise}*.json`.

Table 9: Full LeWM-base diagnostics across four tasks.

Layer / Metric	TwoRoom	PushT	Reacher	Cube	Unit / note
Encoder geometry					
`clean_nn_cos_dist_median`	0.0449	0.2360	0.0633	0.1856	cosine distance
`clean_pair_cos_dist_median`	0.9904	1.0228	1.0252	1.0193	pair-wise cos dist
`clean_effective_rank`	47.60	76.42	61.04	73.25	effective rank
Noise sensitivity					
`noise_angle_deg_median` (@std= 0.005)	5.51°	1.33°	3.22°	1.40°	median angular shift
`noise_to_nn_cos_ratio_median`	0.1031	0.0011	0.0249	0.0016	noise/NN cos ratio
`robust_radius_std`	0.0142	0.0537	0.0142	0.0356	critical noise std
`first_risk_std`	>0.08	>0.08	>0.08	>0.08	first high-risk std
`noise_angle_slope_deg_per_std`	1085.8	284.8	831.7	327.0	°/std angular gain
`geometry_flag`	balanced	robust	balanced	robust	geometry label
Task resolution					
`transition_resolution_ratio_l2`	0.7216	0.3015	0.3704	0.4847	L2 resolution ratio
`transition_resolution_ratio_cos`	0.5538	0.0868	0.1351	0.2347	cos resolution ratio
`id_probe_r2`	0.2889	0.7739	0.1621	0.6657	linear probe R^2
`id_probe_r2_min`	0.2599	0.6786	0.1366	0.0972	min probe R^2
`lidar_rank`	46.06	13.95	45.90	42.46	LiDAR rank proxy
Action effect					
`action_mean_pred_shift_norm`	0.5329	0.1283	0.2518	0.2364	action-perturb mean shift
`action_perturb_pred_shift_corr`	0.2847	0.2873	0.4042	0.2559	shift × $\|a\|$ corr
Predictor stability					
`predictor_rollout_T8_l2`	18.62	18.65	15.17	20.20	T=8 rollout L2 drift
`predictor_target_to_nn_cos_ratio_at_max_std`	1.51e-4	3.54e-6	2.67e-5	3.39e-6	max std target/NN ratio
Latent noise					
`cka_linear_at_max_std`	0.1986	0.5536	0.3085	0.1814	CKA clean vs noisy
`latent_cost_surface_slope_z`	635.31	1.3886	599.45	0.6208	goal latent cost slope

C Heteroscedastic-loss formulation

The scale-preserving heteroscedastic NLL referenced in Section 5.5 (Limitation 4) and detailed in Appendix D is:

$$\mathcal{L}_{\text{hetero}} = \tfrac{1}{2}\,\exp(-s_t)\,\|z_{t+1} - \hat{z}_{t+1}\|^2 + \tfrac{1}{2}\,s_t\,, \tag{2}$$

where s_t is the σ-head-predicted log-variance. During training $\exp(-s_t)$ acts as an automatic weight: high-error transitions are down-weighted and low-error transitions are up-weighted. When $s_t \equiv 0$ the loss reduces to plain MSE.

D Heteroscedastic-loss negative result (data)

This appendix records the full data behind Section 5.5 Limitation 4. The σ-head is trained jointly with the predictor; gradient is detached on the σ path so the mean prediction path is exactly LeWM MSE when σ is constant.

Table 10: Heteroscedastic-loss evaluation.

Task / model	Clean	goal 0.05	pixels 0.05	px+goal 0.05	goal 0.08	px+goal 0.08
TwoRoom LeWM-base	94.00	73.33	72.33	61.33	58.67	50.00
TwoRoom LeWM+noise point-best	98.33	98.00	98.33	98.00	98.67	98.67
TwoRoom hetero	**99.67**	85.33	96.67	84.67	73.33	55.33
PushT LeWM-base	86.33	38.00	17.00	12.00	11.33	4.67
PushT LeWM+noise point-best	**89.33**	87.67	88.00	88.33	89.67	87.00
PushT hetero	**13.33**	7.67	7.67	7.67	9.67	6.00

TwoRoom hetero reaches 99.67% clean — consistent with the prior that low-dimensional discrete tasks benefit from stronger invariance / clustering — but lags noise training on high-noise robustness. **PushT hetero clean is** 13.33% **— a method-level failure**, not a robustness trade-off.

Table 11: Representation diagnostics of the hetero-loss failure.

Metric	TwoRoom base	TwoRoom hetero	PushT base	PushT hetero
`clean_nn_cos_dist_median`	0.0449	0.0281	0.2360	0.1051
`clean_effective_rank`	47.60	33.59	76.42	42.85
`transition_resolution_ratio_cos`	0.5538	0.3780	0.0868	0.0101
`transition_resolution_ratio_l2`	0.7216	0.6055	0.3015	**0.1023**
`id_probe_r2`	0.2889	−0.0573	0.7739	**0.2678**
`action_mean_pred_shift_norm`	0.5329	0.4482	0.1283	0.0841
`predictor_rollout_T8_l2`	18.62	17.90	18.65	14.01

Hetero-loss compresses the representation in both tasks. For TwoRoom (low-dimensional, discrete, redundant), compression is acceptable. For PushT, `transition_resolution_ratio_l2` collapses from 0.3015 to 0.1023 and `id_probe_r`^{2} from 0.7739 to 0.2678 — task-relevant state information is erased. The drop in `predictor_rollout_T8_l2` is not good news either: the latent has become more *predictable* without being more *controllable*.

Mechanism. The σ-head correctly learns per-transition difficulty (high σ on contact frames in PushT, on doorway crossings in TwoRoom, etc.), but using σ as an automatic weight to down-weight high-error transitions misclassifies the contact-and-control-critical states of PushT as "unimportant" and erases them. This is a direct demonstration of the broader trade-off lesson: in contact-heavy control, **hard $\neq$ unimportant**.

This finding motivates a follow-up direction we are currently exploring: per-token *consistency* (not loss-reweighting) controllers driven by detached difficulty signals, which preserve the mean prediction path's gradient distribution. That work is independent of this paper's diagnostic study and will be reported separately.

E One-off cost-swap sanity check

This appendix records the eval-only cost-swap ablation referenced in Section 4.6.1. It is **not** part of the 36-checkpoint canonical evaluation table and uses a separate clean reference (num_eval = 300), so we report it only as a sanity check rather than a pooled statistic.

Table 12: One-off eval-only cost swap on a TwoRoom checkpoint, std = 0.03 pixels+goal.

Variant	Cost type	Cost space	Success rate
A (default)	cosine	normalized	36.0
B (swap)	mse	raw	42.0
Reference: separate clean eval of the same checkpoint (num_eval = 300)	—	—	69.7

Swapping cost recovers only +6 pt (36 → 42), still far below the clean reference (69.7). The narrow reading is therefore: **a sanity check suggests the cost function alone is unlikely to explain the OOD collapse.**